# MNIST Digit Classification Project Report

Manual Implementation of MLP and CNN Models

Linn Erle Kloefta

Lklfta1@student.gsu.edu

CSC4851 Spring 2026

April 20. 2026

# Contents

**1 Introduction** **2**
1.1 Background and Motivation . . . . . 2
1.2 Problem Definition . . . . . 2
1.3 System Overview . . . . . 2

**2 Methodology** **2**
2.1 Implementation Setup . . . . . 2
2.2 Data Pipeline and Preprocessing . . . . . 2
2.2.1 Multilayer Perceptron (MLP) Implementation . . . . . 3
2.2.2 Convolutional Neural Network (CNN) Implementation . . . . . 3
2.3 Concrete Training Walkthrough . . . . . 4

**3 Evaluation** **4**
3.1 Experimental Settings . . . . . 4
3.2 Training Progression and Feature Extraction . . . . . 4
3.3 Qualitative Profiling and Confusion Analysis . . . . . 5

**4 Learning Outcomes** **6**
4.1 Tasks Accomplished . . . . . 6
4.2 Challenges and Debugging Realities . . . . . 7
4.3 Lessons Learned . . . . . 7

# 1 Introduction

## 1.1 Background and Motivation

MNIST digit classification is a common way to test how neural networks learn. Since this was an individual project, I used it as a chance to understand that learning process myself. The assignment required manual implementation, so instead of using high-level deep-learning layers, I implemented forward propagation, loss calculation, and backpropagation directly using NumPy.

## 1.2 Problem Definition

The main task is to classify $28 \times 28$ grayscale images of digits (0–9). In this report, I compare the two required architectures based on how they train, how well they perform on test data, and where they still make mistakes.

## 1.3 System Overview

I study two manually implemented models:

1. **Multilayer Perceptron (MLP):** a 2-layer fully connected network with Sigmoid in the hidden layer and Softmax in the output layer, trained for up to 30 epochs.
2. **Convolutional Neural Network (CNN):** one valid-mode convolution layer with ReLU, followed by a fully connected Softmax output layer, trained for up to 5 epochs.

For both models, I use the same MNIST train/test split and evaluate them using training logs and the figures included in this report.

# 2 Methodology

## 2.1 Implementation Setup

I implemented the two models in separate files: `MLP_template.py` for the fully connected network and `CNN_template.py` for the convolutional network. In both files, the training logic is written manually: forward pass, loss calculation, backward pass, and parameter update.

## 2.2 Data Pipeline and Preprocessing

The same data pipeline is used in both templates, and it is implemented in three steps:

1. Load MNIST from `./data/mnist` using torchvision dataloaders.
2. Convert images with `ToTensor()`.
3. Normalize with `Normalize((0.5,), (0.5,))` so inputs are centered around zero.

Centering inputs around zero helps keep activations and gradients more stable during training.

Batch size is model-specific in the templates:

- **MLP batch size = 128:** gives efficient matrix updates and smoother gradients from batch averaging.
- **CNN batch size = 64:** keeps training stable while limiting the cost of manual convolution loops in NumPy.

### 2.2.1 Multilayer Perceptron (MLP) Implementation

The MLP implementation in `MLP_template.py` follows this step-by-step flow:

1. Flatten each $28 \times 28$ image into a 784-dimensional vector.
2. Run the first linear layer and Sigmoid activation.
3. Run the second linear layer and Softmax to get class probabilities.
4. Compute cross-entropy loss, backpropagate gradients, and update weights with gradient descent.

The forward equations are:

$$Z^{[1]} = XW^{[1]} + B^{[1]}, \quad A^{[1]} = \sigma\left(Z^{[1]}\right) = \frac{1}{1 + e^{-Z^{[1]}}} \tag{1}$$

$$Z^{[2]} = A^{[1]}W^{[2]} + B^{[2]}, \quad A^{[2]} = \text{softmax}\left(Z^{[2]}\right) \tag{2}$$

Backpropagation uses Softmax–cross-entropy output gradients and Sigmoid chain-rule terms. The chosen MLP parameters are:

- **Input size** 784**:** required by flattening $28 \times 28$ images.
- **Hidden size** 128**:** gives enough model capacity without adding too much extra cost; Figure 5 shows only small gains beyond this point.
- **Output size** 10**:** fixed by MNIST class count.
- **Learning rate** 0.5**:** chosen so the MLP learns quickly in early epochs with Sigmoid activation.
- **Weight init scale** 0.05**:** avoids very large pre-activation values that can saturate Sigmoid.
- **Loss epsilon** $10^{-15}$**:** prevents invalid $\log(0)$ in cross-entropy.
- **Epochs** 30**:** assignment-compliant training budget and sufficient for convergence.

### 2.2.2 Convolutional Neural Network (CNN) Implementation

The CNN implementation in `CNN_template.py` also follows a manual step-by-step flow:

1. Keep input images in 2D form and apply one valid-mode convolution filter.
2. Apply ReLU to the convolution output.
3. Flatten the feature map and apply a fully connected Softmax classifier.
4. Compute cross-entropy loss, backpropagate through dense and convolution layers, and update parameters.

This gives the following structure:

- **Convolution stage:** one filter with kernel size 7 and ReLU nonlinearity.
- **Classification stage:** flattened feature map projected to 10 classes with Softmax output.

The chosen CNN parameters are:

- **Input size** 28**:** keeps the spatial image layout explicit for valid-mode convolution.

- **One convolution kernel:** matches the required single-convolution-layer architecture and keeps implementation manageable.
- **Kernel size** 7**:** selected from tested candidates as the best-performing setting under the fixed 5-epoch budget (Figure 6).
- **Flattened size** 484**:** follows directly from valid convolution, $(28 - 7 + 1)^2 = 22^2$.
- **Output size** 10**:** fixed by MNIST class count.
- **Learning rate** 0.05**:** lower than MLP to keep CNN updates stable with manual convolution and dense projection.
- **Loss epsilon** $10^{-8}$**:** prevents invalid logarithms in CNN cross-entropy.
- **Epochs** 5**:** assignment-compliant training budget.

### 2.3 Concrete Training Walkthrough

Figure 1 shows one failed-guess case from my manual training pipeline. The sequence is forward pass, cross-entropy loss, backward pass, and parameter update. I include this case because it clearly shows how a wrong prediction produces a stronger correction signal, which then pushes probability toward the true class in the next update.

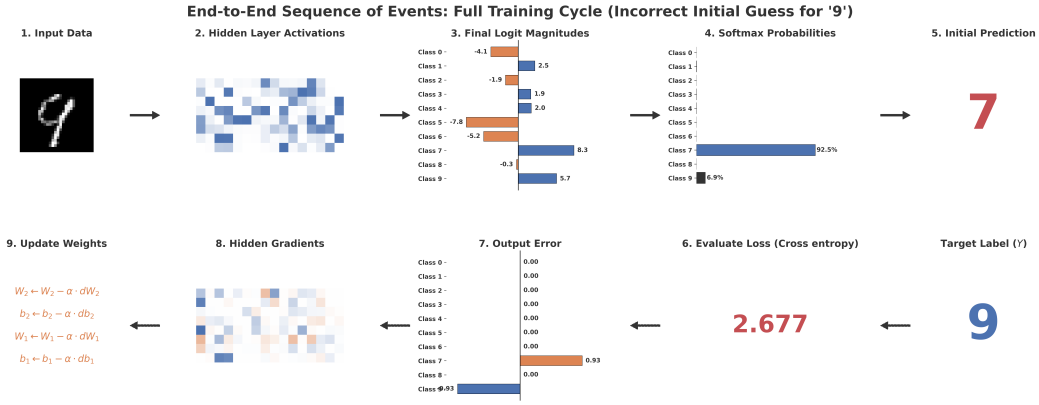


Figure 1: Manual MLP training flow for a failed guess and correction.

## 3 Evaluation

### 3.1 Experimental Settings

Experiments follow the exact training loops in the template files. MLP is trained with hidden size 128, learning rate 0.5, batch size 128, and 30 epochs; CNN uses one $7 \times 7$ valid convolution kernel, learning rate 0.05, batch size 64, and 5 epochs.

### 3.2 Training Progression and Feature Extraction

From Figure 3, I see that the MLP improves quickly at the start and then keeps improving more slowly across later epochs. In this run, test accuracy rises from 0.920 (epoch 1) to 0.978 (epoch 30), while training accuracy reaches 0.999.

Figure 4 shows different behavior for CNN. Training loss decreases steadily across all 5 epochs (from 0.375 to 0.210), but test accuracy follows an uneven path, peaking at epoch 4

(0.947) before dropping to 0.932 at epoch 5. To me, this suggests the CNN is still learning but exhibits less stable generalization than the MLP, likely due to the shorter 5-epoch training budget.

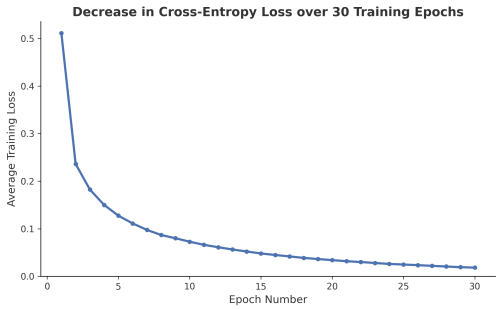


Figure 2: MLP training loss curve.

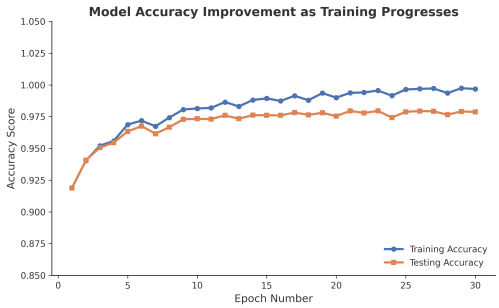


Figure 3: MLP train/test accuracy curve.

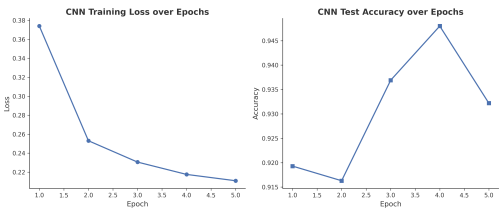


Figure 4: CNN training loss and test accuracy curves.

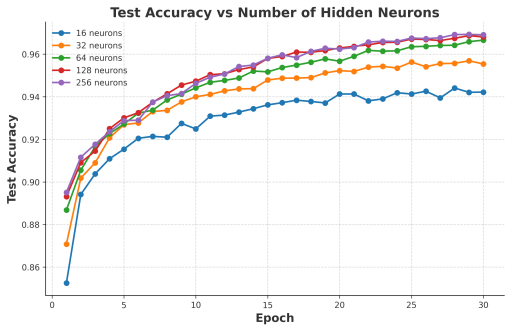


Figure 5: MLP hidden-size benchmark.

Figure 5 shows that larger hidden size helps, but the gain after 128 neurons is negligible (both 128 and 256 neurons reach 0.967 test accuracy by epoch 30). I interpret this as a diminishing-returns point for the current training setup.

Figure 6 summarizes kernel-size behavior for CNN under the 5-epoch budget. Among the tested settings, $7 \times 7$ achieves the best test accuracy (0.933) with competitive loss (0.275), so it is the most reasonable choice under this constraint.

In Figure 7, different hidden units appear to focus on different stroke patterns. Some emphasize vertical or horizontal strokes, while others capture curved or crossing parts of digits. This suggests the hidden layer learns useful parts that the output layer later combines for final class decisions.

## 3.3 Qualitative Profiling and Confusion Analysis

Figure 8 and Figure 9 show a clear difference in error patterns. The MLP matrix is concentrated on the diagonal with few large off-diagonal mistakes, while the CNN matrix shows more scattered errors. From the MLP matrix, notable errors include $2 \to 7$ (8 cases) and $4 \to 9$ (17 cases); from the CNN matrix, larger confusion appears in $4 \to 9$ (45 cases), $8 \to 5$ (49 cases), and $3 \to 5$ (25 cases). The MLP achieves 9768 correct out of 10000 (0.9768) while CNN achieves 9296 correct (0.9296). I read this as the CNN being under-trained at its 5-epoch budget, especially on visually similar digit pairs.

A key limitation is that the training budgets are not equal (30 MLP epochs versus 5 CNN epochs, following assignment rules). Because of that, the performance gap should be

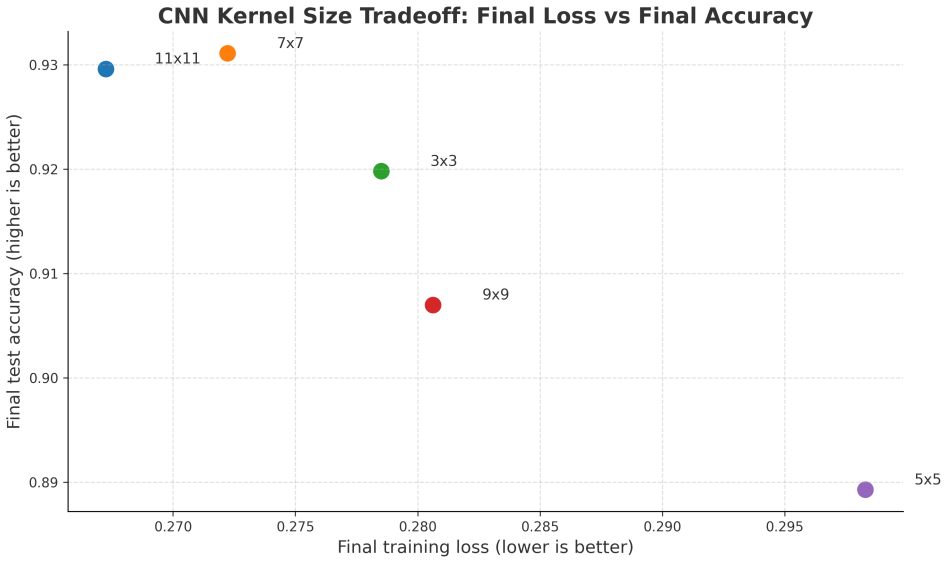


Figure 6: CNN kernel-size tradeoff between final training loss and final test accuracy.

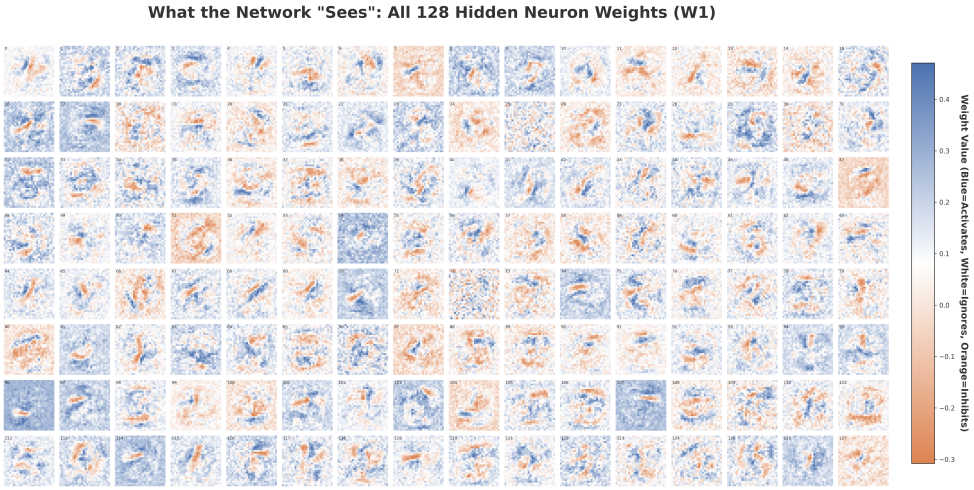


Figure 7: Learned first-layer MLP weights for 128 hidden neurons.

read as a comparison under required settings, not as a final statement about each model's full potential.

# 4 Learning Outcomes

## 4.1 Tasks Accomplished

I completed manual implementations of both required models and their full training/evaluation pipelines in `MLP_template.py` and `CNN_template.py`. The main parts I finished were:

- Manual cross-entropy loss with numerical-stability fixes (max-shift Softmax and an $\epsilon$ term in the logarithm).
- Manual backpropagation for both models. For MLP, I computed Sigmoid-layer derivatives and updated parameters from those gradients. For CNN, I propagated gradients

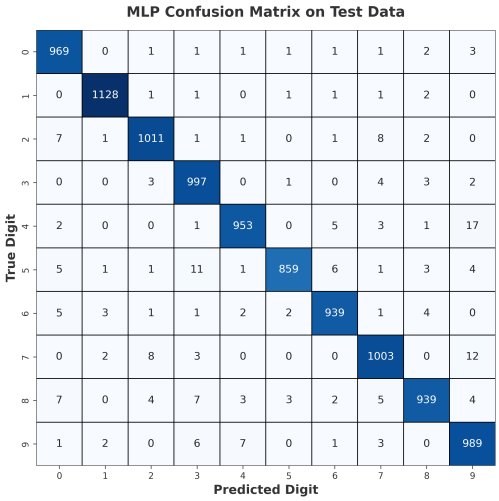


Figure 8: MLP confusion matrix.

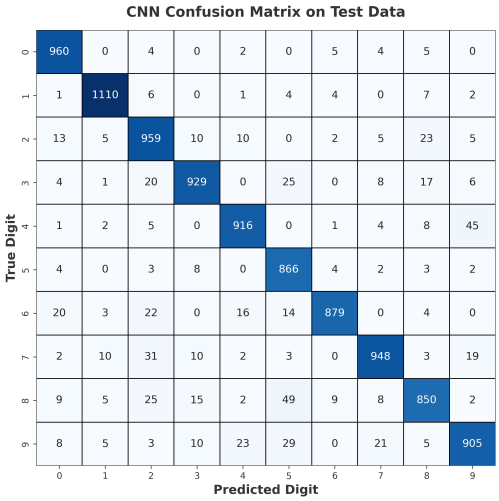


Figure 9: CNN confusion matrix.

from the dense output layer back through flattening and then into the convolution kernel.

- Debugging CNN tensor shapes so each step matched correctly. I checked that convolution outputs, flattened vectors, and reshaped gradients all stayed compatible during both forward and backward passes.
- Targeted hyperparameter checks across the values I benchmarked, including MLP hidden size, MLP initialization scale, and CNN kernel size, to justify the final settings used in the templates.

## 4.2 Challenges and Debugging Realities

The main challenge was debugging the manual training logic without autograd to catch mistakes for me. I had to check the forward pass, the gradients, and the parameter updates by hand, and the failed-guess training flowchart (Methodology, Figure 1) was one of the investigative steps I used to confirm that the correction signal was behaving as expected.

The CNN side was also constrained by the fixed 5-epoch limit in the project. That meant there was little room for slow convergence, so I had to keep the implementation simple, check the tensor shapes carefully, and use kernel-size benchmarking to choose a setting that learned reasonably well within the short training budget.

## 4.3 Lessons Learned

These experiments helped me understand how architecture choices and training budget affect results:

- The final MLP setting (hidden size 128, learning rate 0.5, 30 epochs) reached 0.9768 test accuracy. Hidden-size testing showed negligible gains beyond 128 neurons (both 128 and 256 reached 0.967), so that represents a practical capacity point for this setup.
- CNN test accuracy moved nonlinearly across epochs ($0.920 \rightarrow 0.915 \rightarrow 0.938 \rightarrow 0.947 \rightarrow 0.930$) even though training loss decreased consistently. This pattern reflects the very tight 5-epoch budget, which left little room for stable convergence.
- Kernel-size testing selected $7 \times 7$ as the best option, achieving 0.933 test accuracy under the 5-epoch constraint, confirming its selection in `CNN_template.py`.

- CNN batch-size analysis showed batch 64 performed best (0.9296 final accuracy with 0.2673 loss), validating the chosen batch size of 64 in the template.
- Under the required budgets, the final gap between MLP (97.68%) and CNN (92.96%) is 4.72 percentage points. This gap reflects both the 6-fold longer training duration for MLP and the superior capacity of dense layers versus a single convolutional kernel for feature extraction on MNIST.